Digital Egypt Pioneers Initiative (DEPI)

Final Graduation Project — Data Cleaning & Transformation Documentation

Project Title: UK National Rail Performance & Revenue Analysis
Raw Source File: `railway.csv`
Processed File: `railway Cleaned File.xlsx`
Tool Used: Microsoft Power Query (Excel)
Author: Data Analytics Graduation Project Team

---

1. Project Overview & Data Model Design

Before starting analysis and dashboard creation, I conducted an initial audit of the raw dataset (`railway.csv`). The file contains 31,653 ticket transaction records across 18 columns, tracking passenger purchases, train routes, ticket types, pricing, and arrival status.

To prepare the dataset for reliable reporting and optimize analytical performance, I structured the data using a **Star Schema** dimensional architecture:

- **`Fact_Journey`**: The central transactional fact table containing individual journey details, pricing, adjusted arrival times, and foreign keys.
- **`Dim_Ticket`**: Dimension table detailing ticket class, ticket type, and railcards.
- **`Dim_Route`**: Dimension table detailing unique departure and arrival station pairings.
- **`Dim_Date`**: Continuous calendar table supporting time-based aggregation.

---

2. Comparison: Raw vs. Cleaned Dataset

Area / Column	**Raw State (`railway.csv`)**	**Cleaned State (`railway Cleaned File.xlsx`)**	**Practical Purpose & Justification**
Model Structure	Flat denormalized table (18 columns)	Star Schema (`Fact_Journey` + 3 Dimension tables)	Reduces redundancy and organizes data for reporting.
Data Types	Unassigned / generic text	Typed as `text`, `date`, `time`, and `Int64.Type`	Ensures correct calculations and date filtering.

Area / Column	Raw State (`railway.csv`)	Cleaned State (`railway Cleaned File.xlsx`)	Practical Purpose & Justification
Price	Text format	Defined as whole integer (`Int64.Type`)	Enables direct revenue aggregations.
Railcard	Blank / `None` for non-cardholders	Standardized to `"No Railcard"`	Provides a clear label for standard sales without discount cards.
Reason for Delay	Inconsistent casing and naming (`Signal failure`, `Staffing`, `Weather Conditions`, blanks)	Standardized to `"Signal Failure"`, `"Staff Shortage"`, `"Weather"`, and `"No Delay"`	Eliminates duplicate delay categories and removes blanks.
Station Names	Inconsistent destination naming (`Edinburgh`)	Standardized to `"Edinburgh Waverley"`	Aligns station naming across routes.
Journey Status Logic	Contained records marked `"Delayed"` where scheduled time equaled actual arrival time	Corrected status to `"On Time"` and reason to `"No Delay"`	Ensures delay metrics reflect genuine operational delays.
Missing Actual Arrival	Blank / `null` for cancelled journeys	Categorized in `Actual_Arrival_Status` and imputed with scheduled `Arrival_Time`	Preserves completeness in timestamp columns without data loss.

---

3. Step-by-Step Power Query Applied Steps

A. Fact_Journey Table

Step 1: Ingestion & Header Promotion

I loaded the raw CSV file using standard encoding and promoted the first row to table headers so every column had its appropriate label.

```powerquery
Source = Csv.Document(File.Contents("railway.csv"), [Delimiter=",",
Columns=18, Encoding=1252, QuoteStyle=QuoteStyle.None]),
#"Promoted Headers" = Table.PromoteHeaders(Source,
[PromoteAllScalars=true])
```

Step 2: Assigning Explicit Data Types

I assigned proper data types to each column according to its operational role: dates to `date`, times to `time`, price to `Int64.Type`, and descriptive fields to `text`.

```powerquery
#"Changed Type" = Table.TransformColumnTypes(#"Promoted Headers",{
    {"Transaction ID", type text}, {"Date of Purchase", type date},
{"Time of Purchase", type time},
    {"Purchase Type", type text}, {"Payment Method", type text},
{"Railcard", type text},
    {"Ticket Class", type text}, {"Ticket Type", type text}, {"Price",
Int64.Type},
    {"Departure Station", type text}, {"Arrival Destination", type
text}, {"Date of Journey", type date},
    {"Departure Time", type time}, {"Arrival Time", type time},
{"Actual Arrival Time", type time},
    {"Journey Status", type text}, {"Reason for Delay", type text},
{"Refund Request", type text}
})
```

Step 3: Text Categorization & Standardization

I standardized categorical values to prevent split groups during reporting:

- Standardized casing for `"Signal failure"` to `"Signal Failure"`.
- Consolidated delay reasons: `"Staffing"` to `"Staff Shortage"`, and `"Weather Conditions"` to `"Weather"`.
- Replaced `"None"` in railcards with `"No Railcard"`.
- Standardized `"Edinburgh"` to its full official name `"Edinburgh Waverley"`.

```powerquery
#"Replaced Value" = Table.ReplaceValue(#"Changed Type","Signal
failure","Signal Failure",Replacer.ReplaceText,{"Reason for Delay"}),
#"Replaced Value1" = Table.ReplaceValue(#"Replaced
Value","Staffing","Staff Shortage",Replacer.ReplaceText,{"Reason for
Delay"}),
#"Replaced Value2" = Table.ReplaceValue(#"Replaced Value1","Weather
Conditions","Weather",Replacer.ReplaceText,{"Reason for Delay"}),
#"Replaced Value5" = Table.ReplaceValue(#"Replaced Value2","None","No
Railcard",Replacer.ReplaceText,{"Railcard"}),
```

```
#"Replaced Value4" = Table.ReplaceValue(#"Replaced
Value5","Edinburgh","Edinburgh
Waverley",Replacer.ReplaceValue,{"Arrival Destination"})
```

Step 4: Renaming Columns to Standard Database Conventions

I renamed all column headers from spaced text to consistent `snake_case` naming format for database and data modeling standards.

```powerquery
#"Renamed Columns" = Table.RenameColumns(#"Replaced Value4",{
    {"Transaction ID", "Transaction_ID"}, {"Date of Purchase",
"Date_of_Purchase"},
    {"Time of Purchase", "Time_of_Purchase"}, {"Purchase Type",
"Purchase_Type"},
    {"Payment Method", "Payment_Method"}, {"Ticket Class",
"Ticket_Class"},
    {"Ticket Type", "Ticket_Type"}, {"Departure Station",
"Departure_Station"},
    {"Arrival Destination", "Arrival_Destination"}, {"Date of
Journey", "Date_of_Journey"},
    {"Departure Time", "Departure_Time"}, {"Arrival Time",
"Arrival_Time"},
    {"Actual Arrival Time", "Actual_Arrival_Time"}, {"Journey Status",
"Journey_Status"},
    {"Reason for Delay", "Reason_for_Delay"}, {"Refund Request",
"Refund_Request"}
})
```

Step 5: Handling Empty Delay Reasons

For journeys where trains arrived on schedule, the delay reason field was blank. I replaced empty strings with `"No Delay"` so performance charts can group records without null buckets.

```powerquery
#"Replaced Value3" = Table.ReplaceValue(#"Renamed Columns","","No
Delay",Replacer.ReplaceValue,{"Reason_for_Delay"})
```

Step 6: Logical Status Correction

I audited records flagged as `"Delayed"` where the scheduled `Arrival_Time` equaled the `Actual_Arrival_Time`. To ensure data accuracy, I corrected these journeys:

- Added a temporary evaluation flag `Status Corrected`.
- Updated `Journey_Status` to `"On Time"` and `Reason_For_Delay` to `"No Delay"` whenever the flag was triggered.
- Removed temporary evaluation fields and retained the corrected columns.

```powerquery
#"Added Custom1" = Table.AddColumn(#"Replaced Value3", "Status Corrected", each if [Journey_Status] = "Delayed" and [Arrival_Time] = [Actual_Arrival_Time] then 1 else 0),
#"Changed Type2" = Table.TransformColumnTypes(#"Added Custom1",{{"Status Corrected", Int64.Type}}),
#"Added Custom2" = Table.AddColumn(#"Changed Type2", "Journey_Status.1", each if Text.Lower(Text.From([Status Corrected])) = "1" then "On Time" else [Journey_Status]),
#"Added Custom3" = Table.AddColumn(#"Added Custom2", "Reason_For_Delay.1", each if Text.Lower(Text.From([Status Corrected])) = "1" then "No Delay" else [Reason_for_Delay]),
#"Removed Columns" = Table.RemoveColumns(#"Added Custom3",{"Journey_Status", "Reason_for_Delay"}),
#"Renamed Columns2" = Table.RenameColumns(#"Removed Columns",{{"Journey_Status.1", "Journey_Status"}}),
#"Changed Type3" = Table.TransformColumnTypes(#"Renamed Columns2",{{"Journey_Status", type text}}),
#"Renamed Columns3" = Table.RenameColumns(#"Changed Type3",{{"Reason_For_Delay.1", "Reason_For_Delay"}}),
#"Changed Type4" = Table.TransformColumnTypes(#"Renamed Columns3",{{"Reason_For_Delay", type text}})
```

Step 7: Generating Foreign Key for Ticket Dimension (`Ticket_ID`)

I created a surrogate key combining the first two letters of Ticket Class, Ticket Type, and Railcard (e.g., `St-An-No`) to link the transaction records directly to `Dim_Ticket`. The redundant descriptive columns were then removed from this table.

```powerquery
#"Added Custom4" = Table.AddColumn(#"Changed Type4", "Ticket_ID", each Text.Start(Text.From([Ticket_Class]), 2) & "-" &
```

```
Text.Start(Text.From([Ticket_Type]), 2) & "-" &
Text.Start(Text.From([Railcard]), 2)),
#"Changed Type5" = Table.TransformColumnTypes(#"Added
Custom4",{{"Ticket_ID", type text}}),
#"Removed Columns1" = Table.RemoveColumns(#"Changed
Type5",{"Ticket_Class", "Ticket_Type"}),
#"Removed Columns2" = Table.RemoveColumns(#"Removed
Columns1",{"Railcard"})
```

Step 8: Generating Foreign Key for Route Dimension (`Route_Key`)

I created a clear route identifier joining the departure and destination stations (e.g., `London Euston to Manchester Piccadilly`) to connect the transactions to `Dim_Route`.

```powerquery
#"Added Custom6" = Table.AddColumn(#"Reordered Columns", "Route_Key",
each [Departure_Station] & " to " & [Arrival_Destination]),
#"Changed Type7" = Table.TransformColumnTypes(#"Added
Custom6",{{"Route_Key", type text}})
```

Step 9: Categorizing Actual Arrival Status

I added an operational status column (`Actual_Arrival_Status`) to explicitly label missing arrival times based on journey conditions:

- `"Cancelled"` if the train was cancelled and arrival time was null.
- `"Missing - Investigate"` if the train was not cancelled but lacked an arrival timestamp.
- `"Reported"` for standard recorded arrivals.

```powerquery
#"Added Custom9" = Table.AddColumn(#"Changed Type7",
"Actual_Arrival_Status", each if [Journey_Status] = "Cancelled" and
[Actual_Arrival_Time] = null then "Cancelled" else if
[Actual_Arrival_Time] = null then "Missing - Investigate" else
"Reported"),
#"Changed Type10" = Table.TransformColumnTypes(#"Added
Custom9",{{"Actual_Arrival_Status", type text}})
```

Step 10: Imputing Cancelled Arrival Times & Final Column Ordering

I removed the intermediate flag and station columns (which now reside in `Dim_Route`). For cancelled trips where actual arrival was null, I filled the timestamp using scheduled `Arrival_Time` so the column remains complete, and organized the final column layout.

```powerquery
#"Renamed Columns4" = Table.RenameColumns(#"Changed Type10",{{"Status
Corrected", "Status_Corrected_Flag"}}),
#"Removed Columns3" = Table.RemoveColumns(#"Renamed
Columns4",{"Status_Corrected_Flag", "Departure_Station",
"Arrival_Destination"}),
#"Added Conditional Column" = Table.AddColumn(#"Removed Columns3",
"Actual_Arrival_Tim", each if [Actual_Arrival_Time] = null then
[Arrival_Time] else [Actual_Arrival_Time]),
#"Changed Type1" = Table.TransformColumnTypes(#"Added Conditional
Column",{{"Actual_Arrival_Tim", type time}}),
#"Removed Columns4" = Table.RemoveColumns(#"Changed
Type1",{"Actual_Arrival_Time"}),
#"Renamed Columns1" = Table.RenameColumns(#"Removed
Columns4",{{"Actual_Arrival_Tim", "Actual_Arrival_Time"}}),
#"Reordered Columns1" = Table.ReorderColumns(#"Renamed
Columns1",{"Transaction_ID", "Ticket_ID", "Date_of_Purchase",
"Purchase_Type", "Payment_Method", "Time_of_Purchase", "Price",
"Date_of_Journey", "Departure_Time", "Arrival_Time", "Refund_Request",
"Actual_Arrival_Time", "Journey_Status", "Reason_For_Delay",
"Route_Key", "Actual_Arrival_Status"})
```

B. Dim_Ticket Table

To build the ticket dimension, I referenced the raw dataset, standardized the railcard field, extracted the unique combinations across Ticket Class, Ticket Type, and Railcard, and generated the matching primary key `Ticket_ID`.

```powerquery
Source = Csv.Document(File.Contents("railway.csv"), [Delimiter=",",
Columns=18, Encoding=1252, QuoteStyle=QuoteStyle.None]),
#"Promoted Headers" = Table.PromoteHeaders(Source,
[PromoteAllScalars=true]),
```

```
#"Changed Type" = Table.TransformColumnTypes(#"Promoted
Headers",{{"Railcard", type text}, {"Ticket Class", type text},
{"Ticket Type", type text}}),
#"Replaced Value5" = Table.ReplaceValue(#"Changed Type","None","No
Railcard",Replacer.ReplaceText,{"Railcard"}),
#"Removed Other Columns" = Table.SelectColumns(#"Replaced
Value5",{"Railcard", "Ticket Class", "Ticket Type"}),
#"Removed Duplicates" = Table.Distinct(#"Removed Other Columns"),
#"Renamed Columns" = Table.RenameColumns(#"Removed
Duplicates",{{"Ticket Type", "Ticket_Type"}, {"Ticket Class",
"Ticket_Class"}}),
#"Added Custom" = Table.AddColumn(#"Renamed Columns", "Ticket_ID",
each Text.Start(Text.From([Ticket_Class]), 2) & "-" &
Text.Start(Text.From([Ticket_Type]), 2) & "-" &
Text.Start(Text.From([Railcard]), 2)),
#"Changed Type1" = Table.TransformColumnTypes(#"Added
Custom",{{"Ticket_ID", type text}}),
#"Reordered Columns" = Table.ReorderColumns(#"Changed
Type1",{"Ticket_ID", "Railcard", "Ticket_Class", "Ticket_Type"})
```

C. Dim_Route Table

To build the route dimension, I extracted the station columns, corrected destination naming, created the composite `Route_Key`, and removed duplicate routes to retain unique station pairings.

```powerquery
Source = Csv.Document(File.Contents("railway.csv"), [Delimiter=",",
Columns=18, Encoding=1252, QuoteStyle=QuoteStyle.None]),
#"Promoted Headers" = Table.PromoteHeaders(Source,
[PromoteAllScalars=true]),
#"Replaced Value4" = Table.ReplaceValue(#"Promoted
Headers","Edinburgh","Edinburgh
Waverley",Replacer.ReplaceValue,{"Arrival Destination"}),
#"Renamed Columns" = Table.RenameColumns(#"Replaced
Value4",{{"Departure Station", "Departure_Station"}, {"Arrival
Destination", "Arrival_Destination"}}),
#"Added Custom6" = Table.AddColumn(#"Renamed Columns", "Route_Key",
each [Departure_Station] & " to " & [Arrival_Destination]),
```

```
#"Removed Other Columns" = Table.SelectColumns(#"Added
Custom6",{"Departure_Station", "Arrival_Destination", "Route_Key"}),
#"Reordered Columns1" = Table.ReorderColumns(#"Removed Other
Columns",{"Route_Key", "Departure_Station", "Arrival_Destination"}),
#"Removed Duplicates" = Table.Distinct(#"Reordered Columns1",
{"Route_Key"})
```

D. Dim_Date Table

To support date-based intelligence and filtering, I generated a calendar dimension spanning the earliest to latest journey dates (December 8, 2023 to April 30, 2024) and extracted relevant date parts.

```powerquery
StartDate = #date(2023, 12, 8),
EndDate = #date(2024, 4, 30),
Dates = List.Dates(StartDate, Duration.Days(EndDate - StartDate) + 1,
#duration(1, 0, 0, 0)),
Source = Table.FromList(Dates, Splitter.SplitByNothing(), {"Date"}),
SetDateType = Table.TransformColumnTypes(Source, {{"Date", type
date}}),
AddYear = Table.AddColumn(SetDateType, "Year", each Date.Year([Date]),
Int64.Type),
AddMonth = Table.AddColumn(AddYear, "Month", each Date.Month([Date]),
Int64.Type),
AddMonthName = Table.AddColumn(AddMonth, "Month_Name", each
Date.ToText([Date], [Format = "MMMM", Culture = "en-US"]), type text),
AddQuarter = Table.AddColumn(AddMonthName, "Quarter", each
Date.QuarterOfYear([Date]), Int64.Type),
AddDayName = Table.AddColumn(AddQuarter, "Day_Name", each
Date.ToText([Date], [Format = "dddd", Culture = "en-US"]), type text),
#"Inserted Day" = Table.AddColumn(AddDayName, "Day", each
Date.Day([Date]), Int64.Type),
#"Reordered Columns1" = Table.ReorderColumns(#"Inserted Day",{"Date",
"Year", "Quarter", "Month", "Month_Name", "Day", "Day_Name"})
```

---

4. Final Output Schema Summary

Table Name	Total Columns	Total Rows	Primary Key / Unique Identifier	Foreign Keys
`Fact_Journey`	16	31,653	`Transaction_ID`	`Ticket_ID`, `Route_Key`, `Date_of_Journey`
`Dim_Ticket`	4	25	`Ticket_ID`	—
`Dim_Route`	3	65	`Route_Key`	—
`Dim_Date`	7	145	`Date`	—

---

5. Final Quality Audit

After applying all transformation steps, I verified the resulting data model:

- **Record Integrity:** The fact table contains exactly 31,653 rows matching the original dataset with zero loss of transaction records.
- **Referential Integrity:** Every `Ticket_ID` in `Fact_Journey` links to one of the 25 distinct ticket profiles in `Dim_Ticket`.
- **Clean Categorization:** Zero missing or empty values remain in `Railcard` and `Reason_For_Delay`.
- **Logical Consistency:** Records with equal scheduled and actual arrival times are properly categorized as on-time.
- **Ready for Analysis:** The dataset is fully normalized into a Star Schema, optimized for relationships, DAX measures, and dashboard visualization.